

Professional Certificate Programme in Agentic AI and Applications



Learning & Adaptation in Agents



Revisit the Learning So Far

Let's take a quick look at the key ideas and concepts you have explored in your learning journey.

- How autonomous agents set and pursue goals in dynamic environments
- Real-world applications demonstrating autonomous planning in action
- Key differences between hierarchical and reactive planning approaches
- How planning strategies guide agent actions and adaptations
- Core principles behind autonomous agent decision-making
- How multiple agents coordinate through multi-agent planning strategies
- How reinforcement loops strengthen agent learning and long-term planning

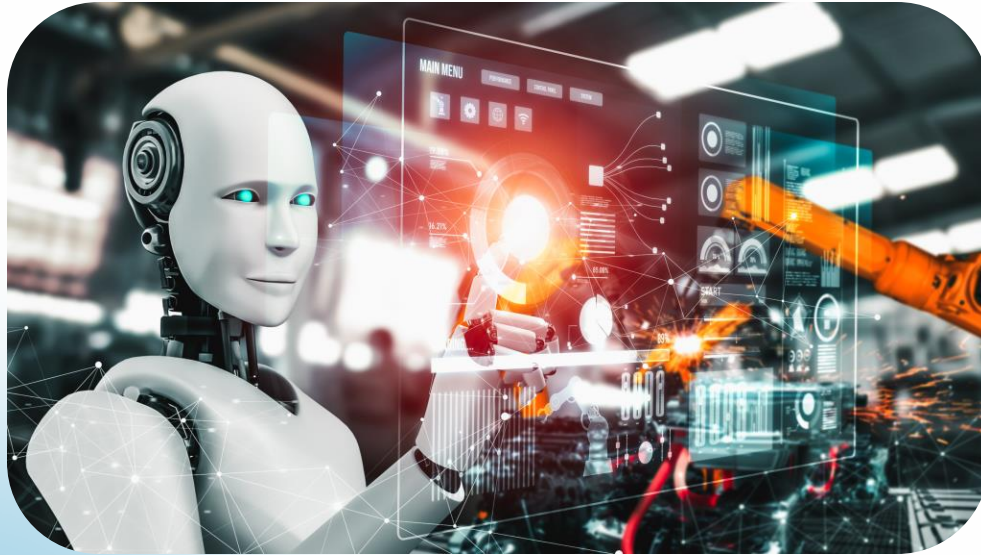
Topics Covered

- Learning & Adaptation in Agents
- Reinforcement Learning
- Reinforcement Learning with Human Feedback (RLHF)
- Introduction to Adaptive Behaviour

Learning & Adaptation in Agents

Learning & Adaptation in Agents

From Hardcoded Rules to Self-Improving Systems



Discover how intelligent agents transcend fixed programming to become truly adaptive systems

Why This Matters

The challenge of static systems

- Rigid logic struggles with new scenarios
- Manual updates increase operational cost
- Static designs cannot handle dynamic data

The promise of adaptive systems

- They learn autonomously from real interactions
- Policies evolve in real time
- System performance improves on a continuous basis

Hardcoded v. Adaptive Agents

Hardcode systems	Adaptive systems
Operate on fixed rules with no flexibility	Learn from feedback and adjust to change
Follow predetermined scripts and cannot adapt to new situations	Evolve their responses based on real-world data and customer interactions

Why this matters:

Customer expectations in e-commerce change rapidly and demand adaptability.

Systems that cannot adapt lead to friction, escalations and missed revenue opportunities.

The Hardcoded Challenge

Traditional E-Commerce Support Limitations



FAQ prison

Can only answer pre-programmed questions and fails on anything outside the script



Change blindness

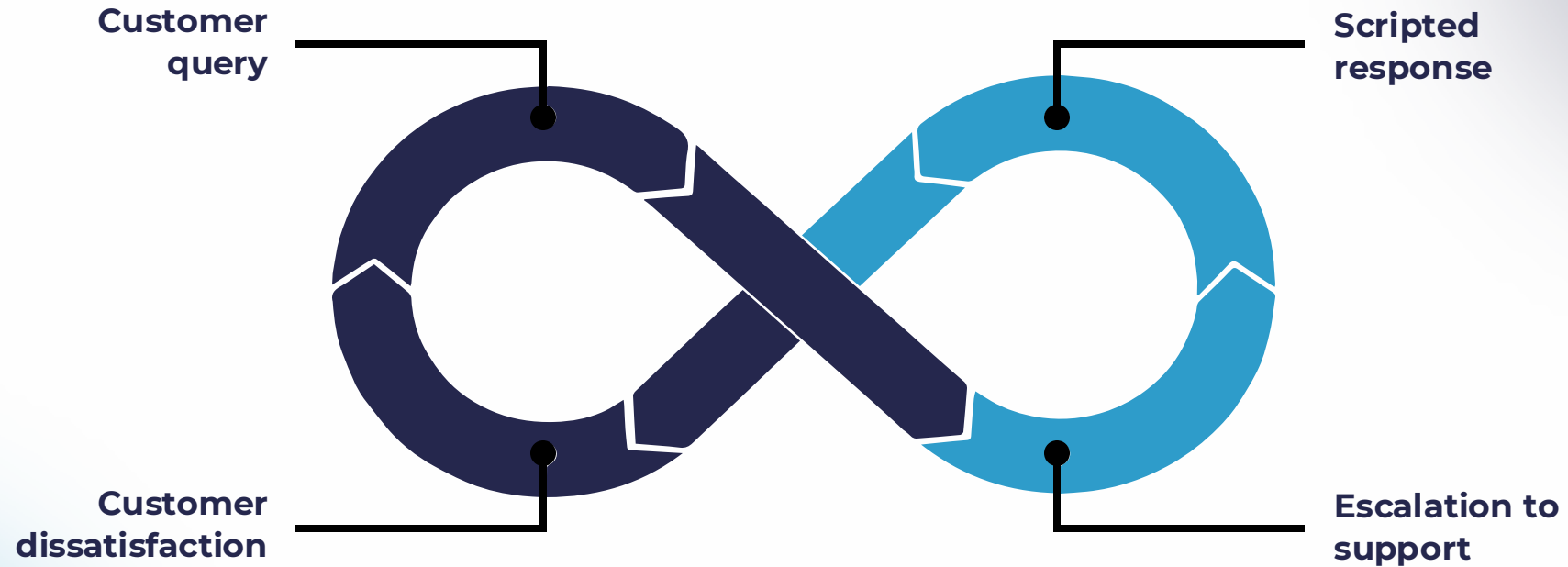
Struggles with new product launches, seasonal changes, and policy updates without manual intervention



Escalation spiral

Has difficulty with nuanced requests, causing frustrated customers and overwhelmed support teams

The Hardcoded Challenge



Bottom line

Rigid systems create customer friction and drive up support costs through unnecessary escalations.

The Adaptive Advantage

Smart E-Commerce Support that Evolves

Analyses chat logs, customer feedback and interaction patterns to identify knowledge gaps

Incorporates new information automatically about products, policies and seasonal variations

Delivers relevant, timely answers that adapt to current business conditions and customer needs

Learning engine



Dynamic updates



Contextual responses



The Adaptive Advantage

40%

Higher resolution rate

More customer issues resolved on first contact

60%

Fewer escalations

Reduced burden on human support agents

25%

Better CSAT scores

Improved customer satisfaction ratings

The result

Adaptive systems create a positive feedback loop, producing better responses, happier customers and more efficient support operations.

Reinforcement Learning

Reinforcement Learning (RL)

Reinforcement Learning is a branch of machine learning in which an agent learns through interaction with its environment.



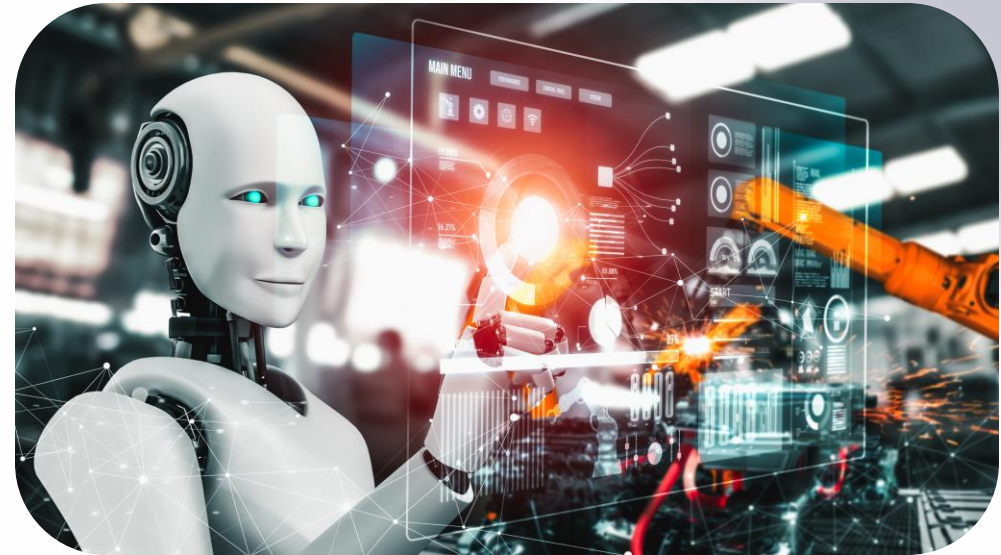
- The agent receives rewards or penalties for its actions and uses this feedback to improve future behaviour.
 - The agent learns, through trial and error, the actions that maximise cumulative reward over time.
-
- It is inspired by behavioural psychology, where learning comes from experience.
 - It works well when supervision is absent and the correct decision is unclear.

Types of Reinforcement Learning



Model-free RL

- Learns directly from experience without building a model of the environment
- **Example:** Q-learning, Deep Q-Networks (DQN)



Model-based RL

- Builds a model of the environment and uses it to plan actions
- **Example:** Monte Carlo methods, Planning-based RL

RL v. Supervised and Unsupervised Learning

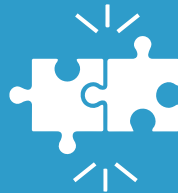
Supervised Learning	Unsupervised Learning	Reinforcement Learning
• Learns from labelled data with input–output pairs	• Learns from unlabelled data by finding patterns without supervision	• Learns from rewards or penalties based on the actions an agent takes • Aims to maximise cumulative reward over time

Significance of RL in AI



Autonomous decision making

Enables systems to make independent decisions by learning from their environment



Exploration and exploitation

Strikes a balance between exploring new actions and exploiting known ones to maximise rewards



Adaptability

Allows systems to adjust to changing environments and improve continually over time

Applications of RL - Robotics



Robot navigation

Enables robots to learn tasks like path planning and obstacle avoidance in unknown environments



Autonomous manipulation

Helps robots to autonomously manipulate objects like picking up items and stacking boxes



Adaptive control

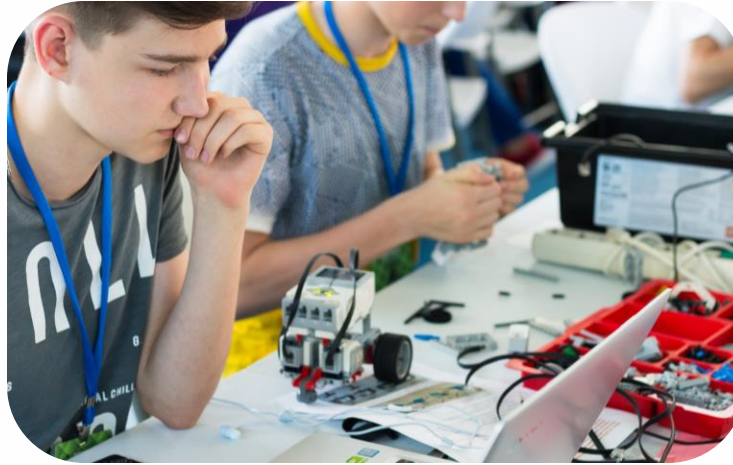
Learns optimal control policies for various tasks such as walking, flying or swimming

Applications of RL - Game Playing



AlphaGo

Defeated world champions in Go using RL, as demonstrated by DeepMind's AlphaGo



Atari Games

Applied RL to control agents in classic Atari games through Deep Q-Learning



Chess and Poker

Transformed game-playing strategies with RL agents like AlphaZero and Libratus

Applications of RL - Autonomous Systems



Self-driving cars

Teaches autonomous vehicles to navigate, make real-time decisions, and enhance safety



Drone navigation

Guides drones in route planning, collision avoidance, and area coverage



Smart cities

Optimises traffic flow, energy consumption and resource allocation

RL in Real-World Systems



Healthcare

Supports personalised treatment planning, advances drug discovery and enables robotic surgeries



Finance

Improves portfolio optimisation, powers algorithmic trading and enhances fraud detection



Energy systems

Optimises power grids, manages energy consumption and strengthens smart grid management

Challenges in RL

Exploration v. exploitation

Balances searching for new strategies with using known successful ones

Requires large amounts of interaction data for an agent to learn effectively

Sample efficiency

Reward shaping

Depends on designing reward functions that correctly encourage the desired behaviour

Struggles in environments where rewards are sparse or significantly delayed

Long-term delayed rewards

Key Components of RL



Agent: Represents the learner or decision maker



Action: Includes choices made by the agent



Environment: Constitutes everything the agent interacts with



Reward: Provides numeric feedback from the environment after each action



State: Depicts a snapshot of the environment at a given time



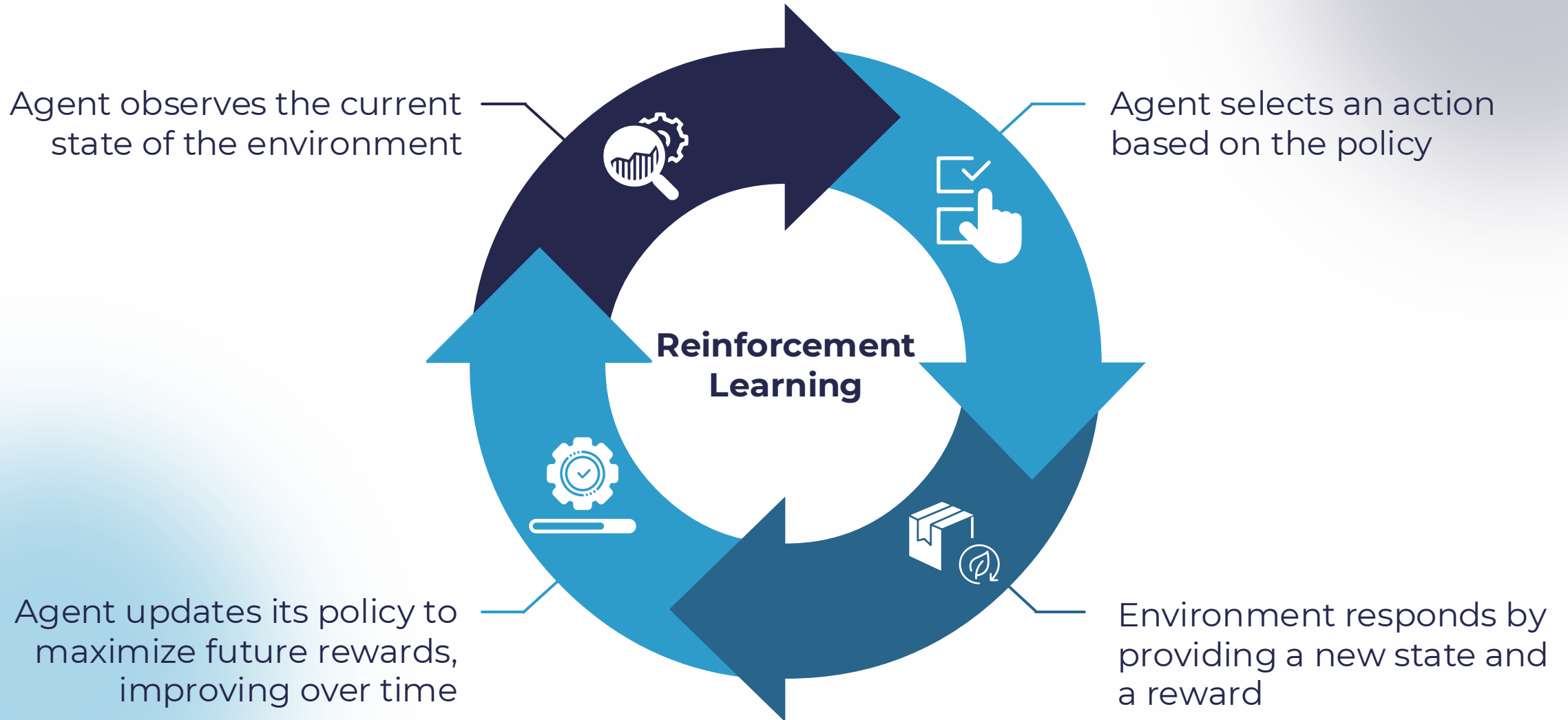
Policy: Defines the strategy an agent follows to map states to actions



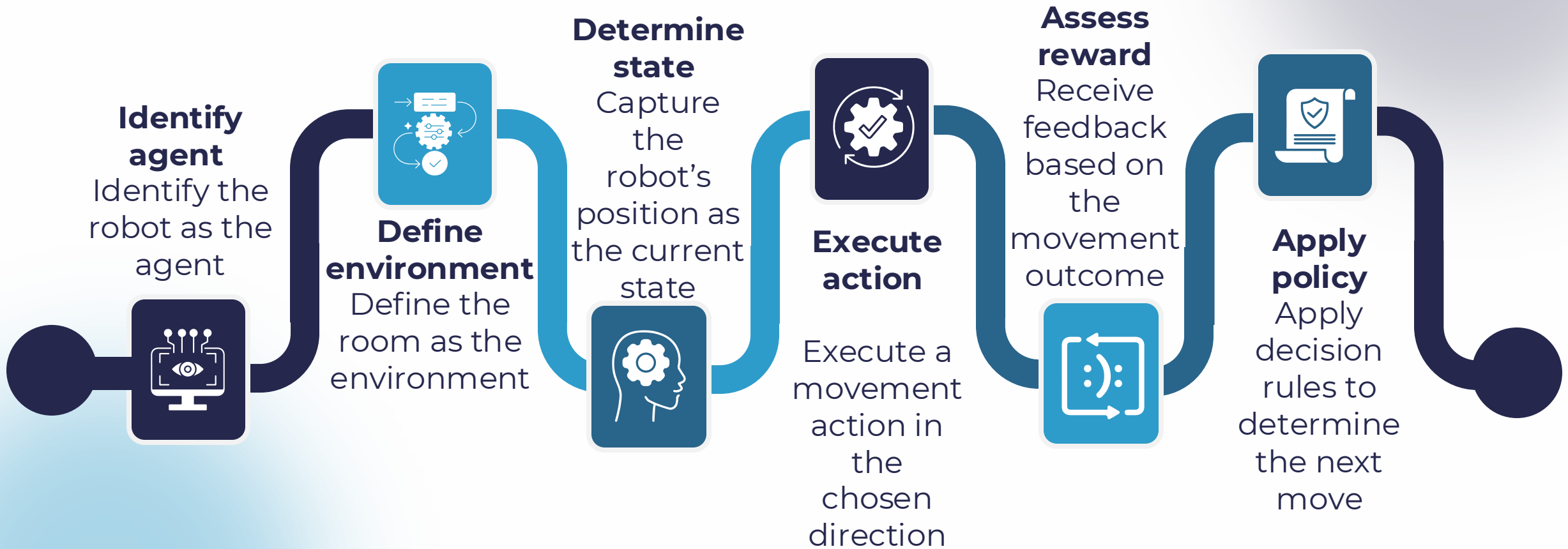
Value function: Measures the long-term benefit of states or actions

How These Concepts Work Together

Interaction Process



Example: A Robot Learns to Move Towards a Goal



Reinforcement Learning with Human Feedback (RLHF)

Reinforcement Learning with Human Feedback (RLHF)

A revolutionary approach that combines reinforcement learning with human judgement to create more aligned and effective AI systems.



Model generates

AI produces multiple output options



Humans evaluate

Users rate and provide feedback



System learns

Feedback becomes training signal



Performance improves

Better alignment with human values

Why RLHF Matters in Agent Design

The Challenge



- Traditional reinforcement learning depends on environmental rewards, but human preferences are far more nuanced and context-dependent.
- Agents must learn to understand these complex human values beyond simple reward signals.

Why RLHF Matters in Agent Design



Reduce harmful outputs

Minimise biased, offensive or inappropriate responses through human oversight



Business alignment

Ensure agents support company goals and ethical standards consistently



Enhanced usability

Improve user experience and satisfaction through continuous human feedback

Key insight: RLHF bridges the gap between what machines optimise for and what humans actually want.

Real-World Impact: Content Moderation Agent

Creates inconsistent moderation by over-flagging harmless content or missing harmful content

Trains agents with human ratings to balance safety with freedom of expression

Delivers higher accuracy, lowers review costs and improves user satisfaction

The problem



RLHF solution



Business results



Real-World Impact: Content Moderation Agent

Before RLHF

- High false positive rates
- Inconsistent policy enforcement
- User complaints about over-moderation

After RLHF

- 35% reduction in false positives
- Consistent, context-aware decisions
- Improved user trust and engagement

Understanding Reward Systems in RLHF Agents

Effective RLHF implementations use varied human feedback signals to create comprehensive learning experiences.

Explicit feedback

Direct human ratings such as likes, stars or rankings provide clear quality signals

- Binary preferences (A v. B comparisons)
- Likert scale ratings (1-5 stars)
- Written explanations and corrections

Implicit signals

User behaviour patterns show preferences through natural interactions without explicit ratings.

- Time spent reading/engaging with content
- Click-through rates and navigation patterns
- Return usage and session frequency

Understanding Reward Systems in RLHF Agents

Effective RLHF implementations use varied human feedback signals to create comprehensive learning experiences.

Task completion

Objective outcomes show if the agent achieved the goal or solved the user's problem

- Query resolution success rates
- Task completion within expected timeframes
- Customer satisfaction post-interaction

Designing Effective Reward Systems for AI Agents

- Agents learn by maximising rewards instead of following rigid, predefined rules
- Rewards act as behavioural signals that guide learning and decision-making



Positive rewards

Encourage and reinforce desired actions, creating learning pathways toward optimal behaviour

Negative rewards

Discourage harmful or inefficient actions, helping agents avoid counterproductive behaviour

Understanding Reward Types

Effective reward systems use multiple feedback signals to guide agents toward desired outcomes.

Direct human feedback through ratings, surveys, thumbs up/down and structured evaluations that provide clear quality signals

User behaviour patterns like clicks, time spent, repeat usage, and abandonment rates that reveal satisfaction without direct feedback

Goal completion metrics like queries resolved, packages delivered or appointments successfully scheduled

Explicit rewards



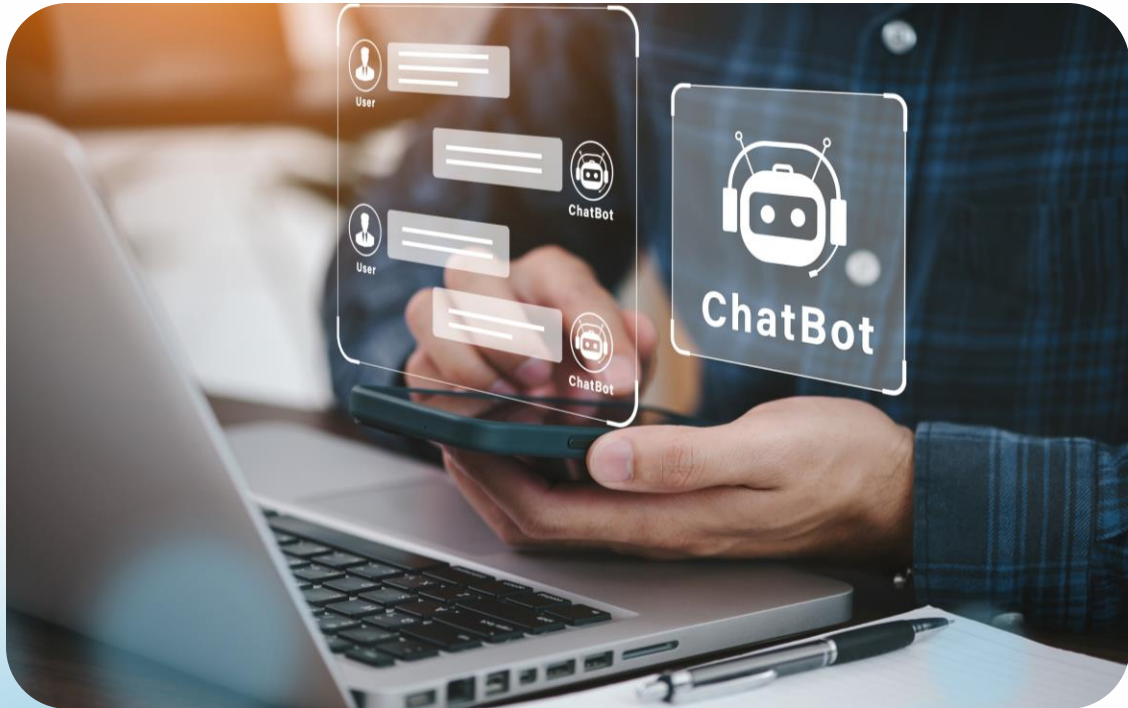
Implicit rewards



Task-driven rewards



Business Example: Customer Support Agent



Positive rewards

- +10 points if resolved without escalation
- +5 points for positive customer rating
- +2 points for fast response time (<30 seconds)

Negative rewards

- -10 points if escalated to human agent
- -5 points for negative customer rating
- -3 points for irrelevant or off-topic responses

Primary goal: Resolve customer queries quickly and effectively while maintaining high satisfaction levels

The Strategic Value of Reward Systems



Drive continuous improvement: Create feedback loops that help agents learn from interactions and continuously enhance performance

Prevent harmful behaviours: Use well-designed negative rewards as guardrails to stop biased, inappropriate or counterproductive actions



Align business goals: Ensure reward structures direct agent behaviour towards business outcomes and user satisfaction

Enable dynamic adaptation: Allow agents to adjust their behaviour as business needs and user expectations evolve



Activity: Designing a Reward System

Introduction to Adaptive Behaviour

Introduction to Adaptive Behaviour

- Traditional systems rely on hardcoded behaviours that are predictable but rigid
- Adaptive systems introduce flexible, intelligent behaviours that evolve with new inputs and changing conditions.



Feedback Loops

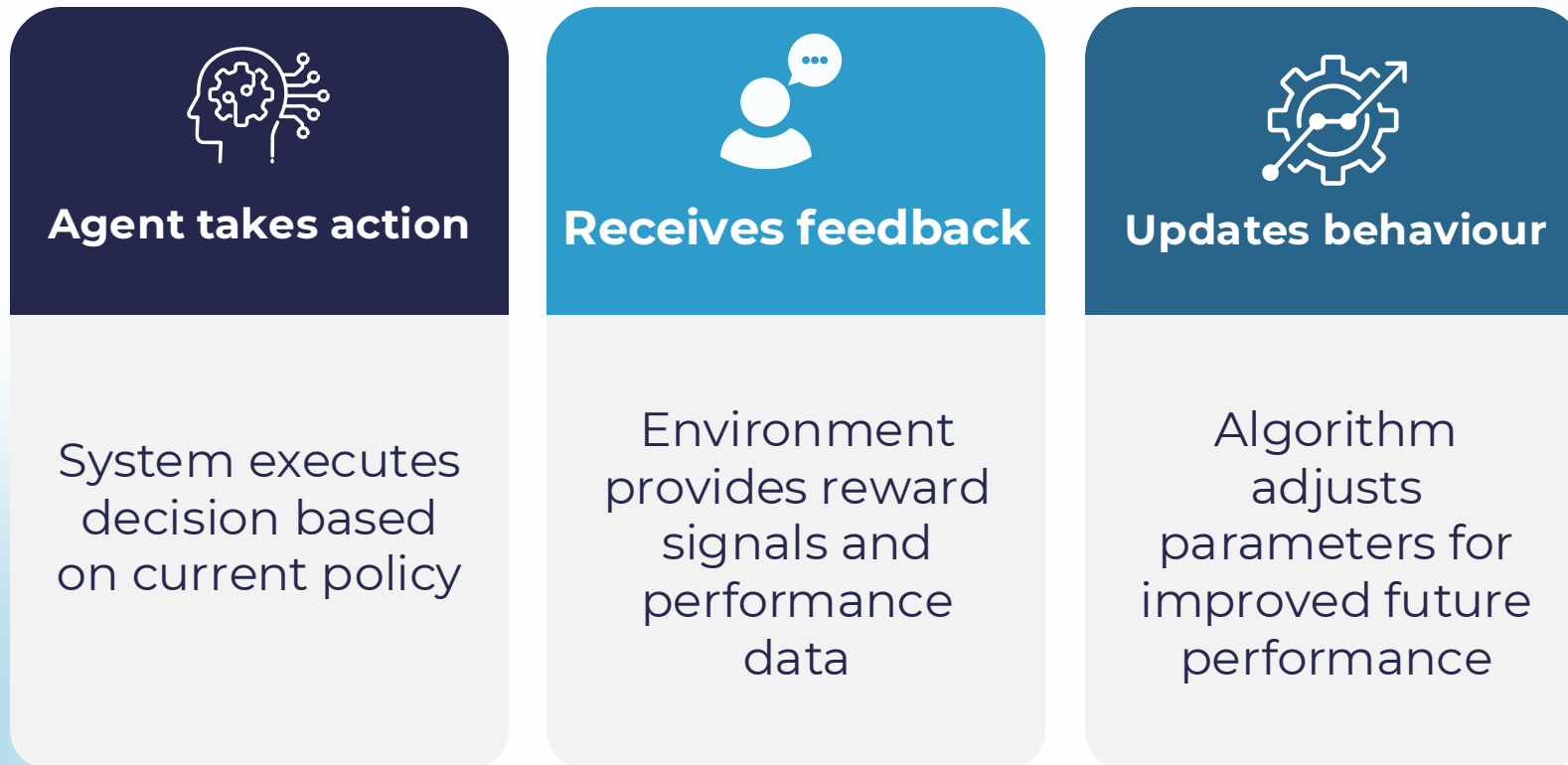
The core mechanism driving continuous improvement through systematic learning cycles

Dynamic environments

Systems that thrive in constantly changing operational conditions

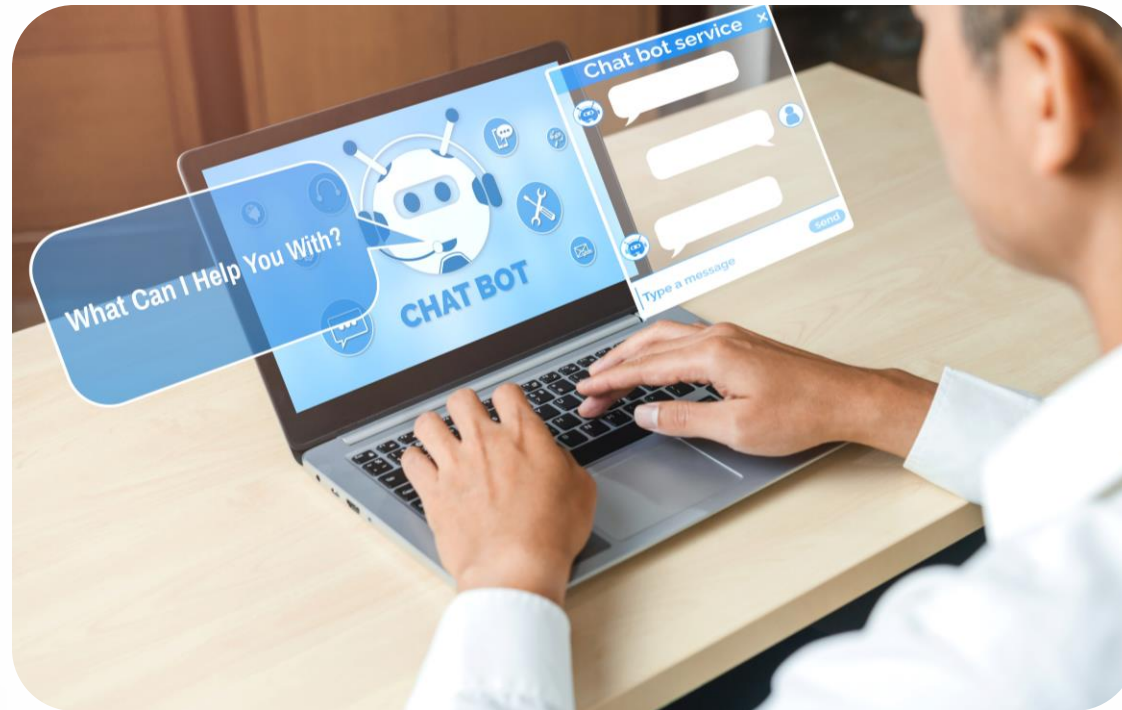
How Adaptive Tuning Works

- Follows a systematic cycle where agents act, receive feedback and update their behaviour
- Creates two distinct learning layers that work together



How Adaptive Tuning Works

Real World Example



Customer chatbots learn from new product FAQs, improving response accuracy and reducing escalations through continuous interaction analysis.

Hardcoded v. Adaptive Systems Comparison

Feature	Hardcoded agent	Adaptive agent
Flexibility	Low (fixed rules)	High (learns from input)
Response to change	Manual reprogramming	Learns via feedback
Scalability	Limited	Continuous improvement
Maintenance cost	High (frequent updates)	Low (self-optimising)
Performance	Degrades over time	Improves with experience

Key insight

Adaptive systems turn operational challenges into learning opportunities, creating sustainable competitive advantages.

Business Example: Logistics Planner



Traditional hardcoded system

- Fixed delivery routes regardless of conditions
- No adaptation to traffic patterns or weather
- Manual intervention required for changes



Adaptive agent solution

- Real-time route adjustments using GPS data
- Learns from delivery delays and patterns
- Automatically optimises for cost and time

Business Example: Logistics Planner

35%

Delivery time reduction

Average improvement
in delivery speed
through adaptive
routing

28%

Fuel cost savings

Operational
efficiency gains
from optimised
fleet management

92%

On-time performance

Reliability
improvement
through predictive
adjustments

Business Example: Logistics Planner

Bottom Line Impact



The adaptive logistics system improved operational efficiency by reducing delivery delays and optimising fleet utilisation.



Its ability to learn from patterns and adjust strategies in real time delivered measurable ROI within the first quarter.

Comparison – Hardcoded v. Adaptive Behaviours

Aspect	Hardcoded behaviours	Adaptive behaviours
Flexibility	Fixed rules, cannot adjust	Learns and adapts to new inputs
Response to change	Requires manual updates/reprogramming	Adjusts automatically via feedback
Scalability	Limited, brittle in dynamic settings	Scales with continuous improvement
Example	FAQ bot fails on new queries	Chatbot learns from feedback to improve

Demos

Journaling Activity – Adaptive Agents in Your Domain

Prompt:

“How can adaptive agents improve workflows in your domain?”

Guiding questions for reflection:

- Where do rigid, rule-based systems slow you down?
- Which tasks could benefit from feedback-driven adaptation?
- What signals (explicit/implicit) could the agent use to improve?
- How would this impact efficiency, accuracy or user satisfaction?

Activity:

- Spend 5 minutes journaling individually.
- Write 2–3 specific use cases in your work or study area.
- Share highlights in small groups or with the class.

Q & A

Thank you